Sampling for Region-Aggregated Spatial Scan Statistics

Foad Namjoo
University of Utah

Drew McClelland
Taptap Send

Michael Matheny
Meta

Jeff M. Phillips
University of Utah

Abstract

Anomaly detection in geospatial data is a crucial tool in geographic information science (GIS), with applications ranging from national security to public-health surveillance to the study of societal disparities. This work focuses on spatial scan statistics and addresses a key mismatch: spatial counts are typically aggregated into predefined regions (census tracts, zip codes, counties), whereas the most efficient scan algorithms operate on spatial point data. The standard remedy—collapsing each region to its centroid, as in widely used tools such as SatScan—is convenient but, as we show, discards the region's spatial extent and causes a significant loss in statistical power. To resolve this, we propose a simple yet scalable fix: replace each spatial region with 20–50 points sampled uniformly from its geometry and spread the region's values evenly across them. This approach improves statistical power while maintaining computational tractability. A convergence analysis explains why so few samples per region suffice. We recommend this sampling-based conversion as the default way to apply point-based spatial scan statistics to region-aggregated data for anomaly detection.

CCS Concepts

• **Information systems → Geographic information systems**; • **Mathematics of computing → Probability and statistics**; • **Theory of computation → Design and analysis of algorithms**; *Sketching and sampling*.

Keywords

Spatial Scan Statistics, Anomaly Detection, Geographic Information Systems, Region Aggregation

ACM Reference Format:

Foad Namjoo, Drew McClelland, Michael Matheny, and Jeff M. Phillips. 2026. Sampling for Region-Aggregated Spatial Scan Statistics. In *Proceedings of The 34th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems (SIGSPATIAL '26).* ACM, New York, NY, USA, 15 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Scan statistics [1, 7, 10, 25] are a fundamental tool in spatial data analysis. They aim to identify spatial regions where a measured value is significantly different from what would be expected based on a baseline distribution. The statistic evaluates this by "scanning" all candidate spatial regions to detect the most anomalous ones. There are many variants of spatial scan statistics [9, 12, 16, 17, 19], and they have been successfully used in various applications such as detecting elevated breast cancer rates [12], monitoring emerging public health threats [6, 18], and analyzing spatial crime patterns [21]. These tools are central to anomaly detection workflows in GIS and public health surveillance.

Scan statistics can be applied to spatial data provided in two formats. The first format represents data as a set of geolocated points $X \subset \mathbb{R}^2$; such as home addresses or GPS-tracked locations. The most common software for these tasks operates on an input of this form, such as SatScan [11]. The second common format aggregates the data into predefined regions (e.g., census tracts, zip codes, counties) [23, 25]. The baseline and measured values can then be accumulated for each region. These aggregations may be necessary due to privacy constraints (an individual's data can be less easily identified if it has been region aggregated) or because data are only available at a coarse spatial resolution.

To apply point-based algorithms to region-aggregated data, it is common practice to convert each region into a single point using its centroid. Although computationally convenient, this modeling choice introduces approximation errors and can significantly reduce the *statistical power*—the ability to reliably detect a true anomaly. In particular, it does not capture the spatial diversity and extent of the underlying region, leading to weaker anomaly detection.

This paper addresses the above limitation and proposes a robust yet scalable alternative. *The proposed solution is simple:* ***instead of representing a region by one centroid point, we sample multiple points (typically 20 to 50) in each region, and uniformly distribute the region's values across those points;*** We show that this approach significantly increases the statistical power of these methods under planted region experiments and demonstrate its scalability on datasets at local, state, and national levels. By bridging region-aggregated data with efficient point-based algorithms, our method improves anomaly detection accuracy and strengthens the reliability of spatial decision-making.

2 Preliminaries

This section reviews both the classical point-based formulation and the challenges that arise when extending it to region-based settings.

Point-Based Spatial Scan Statistics. In the point-based setting, spatial scan statistics operate on a set of spatial points $X \subset \mathbb{R}^2$, where each point $x \in X$ has an associated baseline value $b(x)$, which represents the expected background level of the measured phenomenon at location x (e.g., population or expected case count)

SIGSPATIAL '26, Riverside, CA, USA

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

and a measured value $m(x)$ representing the observed quantity of interest (e.g., people with cancer). The goal is to identify regions where the measured values deviate from the baseline, using a family of geometric shapes $\mathcal{S}$ and a cost function ϕ.

The family $\mathcal{S}$ is typically defined by a geometric shape. Common choices are disks (interiors of circles) or (axis-aligned) rectangles. Each shape $S \in \mathcal{S}$ induces a subset $Y = X \cap S$ of points, and the scan statistic evaluates each induced subset Y using the anomaly score function $\phi(Y)$.

A widely used cost function $\phi(Y)$, is derived under a Poisson model [10] for the distribution of measured values $m(x)$ given a baseline value $b(x)$. An anomalous subset $Y \subset X$ is one that would be well modeled by a different (often larger) rate of measured values than $X \setminus Y$. The final anomaly score $\phi(Y)$ is derived as the negative log-likelihood ratio between the best model with a different rate for Y and $X \setminus Y$ and for the best model where the rate is the same for all X:

$$\phi(Y) = m(Y) \log \frac{m(Y)}{b(Y)} - (1 - m(Y)) \log \frac{1 - m(Y)}{1 - b(Y)},$$

where, $m(Y) = \frac{1}{M} \sum_{x \in Y} m(x)$ and $b(Y) = \frac{1}{B} \sum_{x \in Y} b(x)$, where $M = \sum_{x \in X} m(x)$ and $B = \sum_{x \in X} b(x)$. Although other cost functions exist [2, 11] (e.g., under Bernoulli or Gaussian assumptions), we adopt the Poisson model due to its widespread use and compatibility with both point and region-based inputs.

The final step of spatial scan statistics is to identify the shape S that (approximately) maximizes $\phi(X \cap S)$. This defines the spatial region that is most anomalous, and the statistic itself is

$$\Phi_{\mathcal{S}}(X, m, b) = \max_{S \in \mathcal{S}} \phi(X \cap S).$$

While there are infinite number of geometric shapes that we could scan over, we only need to consider a finite number of them. This is because the function ϕ only depends on which points are contained in the shape, so the number of shapes considered is bounded as a function of the number of input points $n = |X|$. Although there is an exponential number of subsets 2^n, restricting to geometric shapes reduces this to a polynomial number: at most n^3 for disks and at most n^4 for rectangles. Brute-force enumeration of these candidates, spending $O(n)$ to score each one, therefore takes $O(n^4)$ and $O(n^5)$ time for disks and rectangles, respectively; sophisticated algorithms reduce this to near-quadratic $O(n^2 \log n)$ for rectangle discrepancy [3], which the `pyScan` implementation we use in Section 4 exploits.

Region-Based Spatial Scan Statistics. In many real-world datasets, data are aggregated over spatial regions (e.g., counties), rather than available at individual locations. Let Z be a collection of regions, with each region $z \in Z$ associated with the baseline and measured values $b(z)$ and $m(z)$, respectively. Given any subset $Y \subset Z$, where one can define $b(Y)$ and $m(Y)$ analogously, and then the same cost functions $\phi(Y)$ can be employed.

The key challenge is to define the candidate subsets Y (regions of interest). While a potential subset could be any connected set of regions, as in FlexScan [22, 23], this is not tied to geometric shapes, and can potentially over-fit to strangely shaped regions. Also, it can be significantly slower than other scanning algorithms [8]; also see Table 1 in Section 4.6.

A more common strategy is to use geometric shapes $\mathcal{S}$ (e.g., disks, rectangles) as before, but apply them to regions. This presents two main obstacles: First, in the point-based setting, one can identify a canonical shape from a valid subset $Y \subset \mathbb{R}^2$ as the *smallest* shape $S_Y \in \mathcal{S}$ that contains Y (and nothing from $X \setminus Y$). Given a set of points Y and $\mathcal{S}$ as disks or rectangles, the smallest shape can be found easily with computational geometry. However, with region-based input, where each $z \in Z$ has a complex geometry (e.g., a polygon in a shapefile), the computation of the minimal enclosing shape is more difficult. Second, geometric shapes $S \in \mathcal{S}$ are likely to be unable to partition Y from $Z \setminus Y$, meaning that they may partially overlap with regions $z \in Z$. It makes it more difficult to cleanly partition Z into Y and $Z \setminus Y$.

It is not immediately clear how to handle these in defining $b(Y)$ and $m(Y)$, at least not efficiently. One approach is the area-based framework of Buchin et al. [5]. They model the map as a polygonal subdivision with per-region case and population counts, and weight a regions contribution towards ϕ proportional to its overlap. Their method, however, only considers a fixed-scale polygonal window. They discretize the continuous placement problem by constructing the arrangement of combinatorially distinct placements of the polygonal shape with respect to the subdivision, and then optimize the likelihood within each cell. In experiments on real geography with synthetic cases, their area-based methods localized clusters more accurately than centroid-based baselines. They also introduced a non-homogeneous variant that showed lower detection power. In practice, circular windows are approximated by regular polygons. A limitation of their method is the fixed scale and aspect ratio (for rectangles) of anomalous regions they consider.

However, the most widely used heuristic to resolve these difficulties is to simply represent each region $z \in Z$ at a single point x_z at its centroid, assigning $b(x_z) = b(z)$ and $m(x_z) = m(z)$. This allows for direct application of point-based scan algorithms. However, this approximation introduces geometric inaccuracies: a shape S containing x_z may only partially intersect z, leading to a possible misalignment between the shape and the underlying region. This approach *ignores* this potential error.

2.1 Software for Computing Spatial Scan Statistics

Several software tools implement the spatial scan statistic, each with trade-offs between speed, flexibility, and accuracy in detecting region shapes.

Probably the oldest and most widely used implementation is SatScan [11], which supports both point-based and region-based data. For regions, it reduces the regions to single points—typically centroids—and scans over circular or elliptical windows to maximize $\Phi_{\mathcal{S}}$. It offers a variety of statistical models, as well as searching for space-time, purely temporal, and seasonally-defined anomalies.

The pyScan [13] offers a modern Python interface with significantly improved scalability. It relies on a well-engineered C++ backend with algorithmic enhancements for computing spatial scan statistics efficiently. Additionally, pyScan allows for guaranteed approximations of the optimal score, trading small controlled loss in accuracy for substantial gains in speed. In this work, we enable a mild approximation in pyScan; see Section 4.

Another option is FlexScan [22], which operates directly on a region-based input. It performs an exhaustive search over combinations of spatially connected regions to identify clusters. FlexScan allows for more flexibility in cluster shape, but several previous studies have highlighted critical limitations. Tango et al. [23] noted that FlexScan has high computational overhead, especially when applied to datasets with a large number of spatial regions; an efficient algorithm is needed. Moreover, it is limited to clusters of at most 15 regions; if the anomalous area is larger than that, it reports multiple disjoint regions without indicating if they are contiguous or not. In the large cluster case, it does not report a single region or single significance score. We also found that FlexScan requires more manual pre- and post-processing; for instance, if the data set contained unconnected regions, it failed to report any clusters unless these were first removed.

We also obtained a Java (JDK 17) implementation of the area-based method of Buchin et al. [5] by contacting the authors. The method scans a window of a *fixed* scale, as required by the code. Because the spatial extent of an anomaly is a key aspect of what one is trying to discover, this is a strong assumption. The paper recommends trying several scales (using 9 multiples $\{0.5, 0.7, 0.85, 0.95, 1.0, 1.05, 1.15, 1.3, 1.5\}$ of the guess – or given the true planted size) retaining the highest scoring anomaly across all scales, and we follow this in our experiments. The true size would not be known by a practitioner. By contrast the scan methods of SatScan and PyScan that our method extends are able to consider all scales from some shape family intrinsically.

3 Methods

To address the challenge of converting region-aggregated spatial data to be compatible with point-based scan statistics, we propose a principled alternative to the common centroid approximation.

Baseline: Centroid Method. In the centroid-based approach, each region z is represented by a single point x_z located at its geometric centroid. The entire measured and baseline values of the region are then assigned to that point, i.e. $m(x_z) = m(z)$ and $b(x_z) = b(z)$. Although computationally efficient, this reduction discards the internal spatial structure of the region, potentially diminishing statistical power and introducing bias.

Our Proposed Sampling-Based Method. Instead of using one centroid point per region, we propose replacing each region z with a set of k representative points: $x_{z,1}, \ldots, x_{z,k}$. These points are sampled uniformly at random from within the geometry of z. The region's values are then evenly distributed across these points, so that

$$m(x_{z,j}) = \frac{m(z)}{k}, \quad b(x_{z,j}) = \frac{b(z)}{k}, \quad \text{for all } j = 1, \ldots, k.$$

This strategy preserves both the spatial extent and the internal structure of the original region while remaining compatible with existing point-based scan algorithms.

While we explore other variants, we find that random selection of these k points in each region is effective. We explore the values of k from 1 (centroid) to 50. Larger values of k tend to produce higher statistical power, as they better approximate the complete geometry of the region. We find that while the results improve with larger k, the full $k = 50$ points are not always needed. The optimal choice may depend on dataset complexity and computational constraints. Figure 1 shows an example of this sampling-based conversion applied to counties in the state of Arkansas. The left panel uses $k = 10$ points per region, while the right uses $k = 50$.

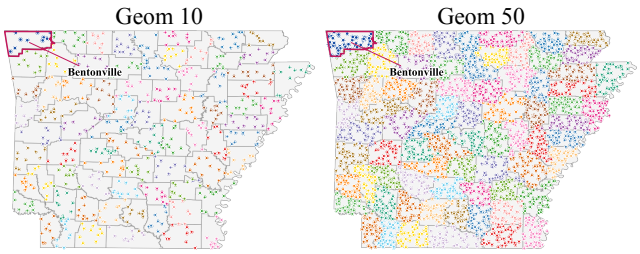

Figure 1: Region-to-point sampling on Arkansas counties: each county is replaced by k points drawn uniformly at random from its polygon. For example Bentonville County (outlined), contains exactly 10 points in Geom 10 (left) and exactly 50 points in Geom 50 (right).

3.1 Recovering Planted Anomalies

To evaluate the statistical power of different scanning algorithms, we consider their ability to recover known anomalous regions that we synthetically "plant" in the data. We do so by generating a shape $S \in \mathcal{S}$ (a "planted region"), and following the Poisson statistical model, randomly generate measured values m, where the rate is higher inside S (rate p) than outside (rate q). We then run various spatial scan statistics algorithms and assess how accurately they recover S by identifying an anomalous region $\hat{S}$. Our experiments primarily use planted rectangular regions and focus on algorithms optimized for detecting rectangular shapes.

We evaluated how well an algorithm works by how closely the identified anomalous region $\hat{S}$ (the *Discovered Rectangle*) matches the planted one S (the *Target Rectangle*). In general, we do not expect these to match exactly due to spatial rounding errors and because we generate values of m randomly by a Poisson process (with rate p or rate q). As the rates p and q become very similar, the most anomalous rectangle from the generated data tends to become less distinct from the generating one.

To quantify the similarity between S and $\hat{S}$, we use a modified Jaccard distance defined on a large set of fixed points A. We construct A by uniformly sampling 500 points within each spatial region in the dataset, resulting in $|A| = 500n$ where n is the number of regions. This dense sampling ensures robust, reproducible overlap comparisons. We define *Point Jaccard distance* as the following:

$$d_{\text{JAC},A}(S, \hat{S}) = 1 - \frac{|A \cap (S \cap \hat{S})|}{|A \cap (S \cup \hat{S})|}.$$

which measures the proportion of sampled points that fall in both S and $\hat{S}$ relative to those in either S or $\hat{S}$.

Figure 2 illustrates an example for Arkansas counties using the Geom 5 method. Lower Jaccard distance values indicate better recovery of the planted anomaly. This provides visual intuition for the meaning of various Jaccard distances. In particular, the last panel shows where the target region is a disk, not a rectangle; it has a Jaccard distance of 0.161. This is especially informative since in

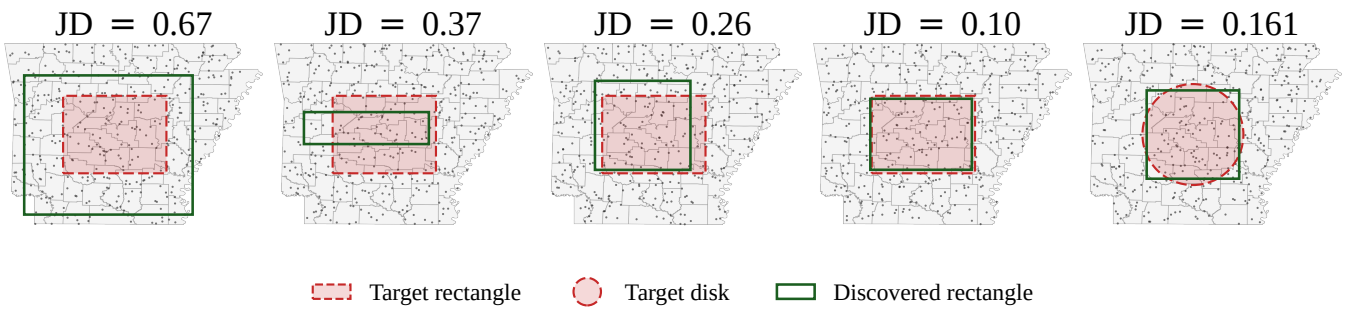

Figure 2: Point Jaccard distance between target (red) and discovered rectangles (green outline) for Arkansas counties. Let last panel shows the best fit discovered rectangle to a circular shape; with Point Jaccard distance of 0.161**, providing an approximate distance floor since the target may not be a rectangle.**

practice the true anomalies my not be rectangular, and this provides a sort of "shape floor" for how similar we should expect to recover a planted anomaly. That is, below about 0.2 Jaccard distance is probably sufficient. On the other hand a Jaccard distance above 0.35 is not a good fit.

4 Experimental Evaluation

Our main experiment evaluates how effectively each scanning method recovers a planted anomalous region under varying conditions. We vary the algorithm's parameters and the experimental setup to see how algorithms perform in different case studies. To capture a diverse range of spatial structures and region complexity, we conducted experiments on six different geospatial datasets. Arkansas, New York City, Utah, California, Georgia, and the entire United States. These cases represent a spectrum from local compact, densely populated regions (e.g., NYC) to large, sparsely populated ones (e.g., Utah).

Each experimental result was produced by taking the average Jaccard distance of the 20 trials. Both the experimental setup and the newly proposed algorithms are random, so it is helpful to understand the distribution of the results. We also plot shaded regions that show the results' average values plus or minus one standard deviation. Methods with narrower bands are more consistent, whereas wider bands indicate a higher sensitivity to randomness in the setup.

Measuring Statistical Power. While there are various ways to formulate it, *statistical power* inherently measures how challenging a problem can be and still be reliably solved with a method. The more challenging the problem reliably solved by a method, the greater the statistical power of that method.

We vary problem difficulty—and thus statistical power—using two factors: the *pq difference* and the *size of the planted region*. The *pq* difference (the primary focus of this paper) captures the contrast between the measured rates inside (p) and outside (q) the planted region. In our experiments, we always set $q = 0.2$, so for each baseline value $b(z)$ outside a planted region, we expect to observe $\mathsf{E}[m(z)] = (0.2)b(z)$ as the corresponding measured value. We vary the rate within the region from $p = 0.2$ (so the *pq* difference is 0) and $p = 0.9$ (so the *pq* difference is 0.7). The more different these values are, the more significant the anomaly should be, and the easier it should be to detect it. For example, when $p = 0.9$, the difference ($p - q = 0.7$) should present an obvious anomaly; when $p = 0.2$, the planted region should be indistinguishable from the background.

A method with greater statistical power can successfully recover the planted region even at lower *pq* differences. Thus, we interpret the lower thresholds of p (with fixed q) at which recovery is still successful as evidence of higher power.

Methods Considered. We evaluated five point-based conversion strategies using the `pyScan` (with adaptive grid at size 100×100) and restrict our scan shapes to rectangles. We chose this shape because it provides the best combination of representational complexity and computational efficiency. We consider a region-based input and convert the problem to point-based input in 5 different ways. Centroid is the baseline approach, where each region is represented by a single point on its geometric centroid. Random Point replaces each region with one randomly sampled from within the region's shape. This is a variation of our proposed method but uses only a single point. This variant shares the same computational cost as the centroid method, but introduces randomness.

Geom 5, Geom 10, and Geom 50 represent our proposed multi-point sampling strategy. They replace each region with 5, 10, or 50 randomly sampled points, respectively, distributing the region's baseline and measured values evenly across those points.

4.1 Zip Codes in New York City

We start our evaluation by studying the zip code boundaries of New York City. These shape files vary significantly in size and geometry – although they are designed to represent zip codes with approximately equal populations. We assign the same baseline value to each zipcode region. We synthetically plant a rectangular region in a central portion of the map, covering approximately one-third of the total area. This planted region is shown inset in Figure 3. Using this shape, we fix $q = 0.2$, and vary the rate p inside the shape.

Figure 3 shows the Point Jaccard distance as a function of the *pq* difference on the x-axis for each method. Both the Centroid and Random Point approaches exhibit similar behavior: they are relatively noisy and consistently identify the planted region only when the *pq* difference is approximately 0.35 or 0.4. In contrast, as we increase the number of random points from 1 to 5, 10, or 50 per

region, the results become more stable and the planted region is detected with much smaller pq differences.

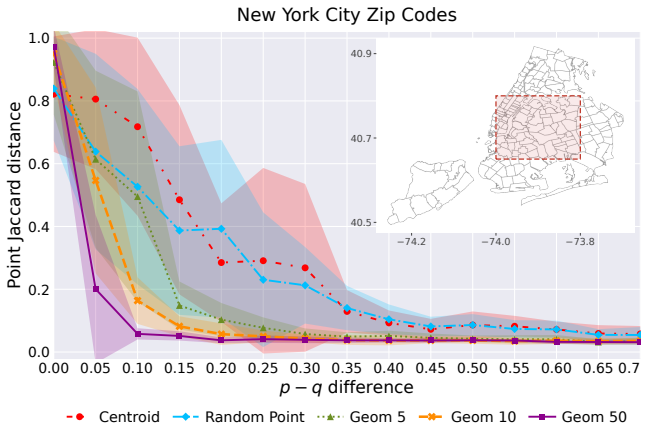

Figure 3: New York City with 263 zip codes. The inset (upper right) shows the planted target rectangle (red dashed), spanning longitudes −74 to −73.8 and latitudes 40.6 to 40.8. The main plot shows Point Jaccard distance versus pq difference for each method at grid resolution 100, averaged over 20 trials with ±1 standard deviation bands.

Specifically, with 5 random points, the planted region is reliably recovered at pq differences of 0.2 or greater; with 10 points, detection occurs at pq differences as low as 0.1; and with 50 points, successful recovery is observed even at a pq difference of 0.05. At this threshold, the Point Jaccard distance drops to between 0.1 and 0.2—indicating substantial overlap—whereas the Centroid and Random Point approaches still yield distances above 0.8, indicating minimal overlap.

Note that even when the pq difference is very large (e.g., at least 0.5), all methods converge to a Point Jaccard distance of around 0.1. This plateau does not converge to 0 due to the inherent stochastic variation in the data generation process under the Poisson model. Although we could potentially devise a way to measure the accuracy of the region recovery so that it converges to 0 error, this lack of convergence to 0 is consistent with the underlying statistical Poisson model on a finite sample.

4.2 Counties of US States

Utah. Next, we consider the 29 counties in the state of Utah; see the inset panel of Figure 4. Compared to regions such as Arkansas (in Figure 1), these counties are fewer in number and often have irregular shapes, which introduces additional geometric challenges for region-based analysis.

The performance of the various methods is shown in Figure 4. Due to the number of regions and their complex shapes, we observe substantial variability in the Point Jaccard distance for both the Centroid and Random Point approaches, as well as for Geom 5. These methods struggle to consistently detect the planted region, particularly at low pq differences.

The Geom 10 and Geom 50 approaches show more reliable behavior. Although some instability persists at small pq differences,

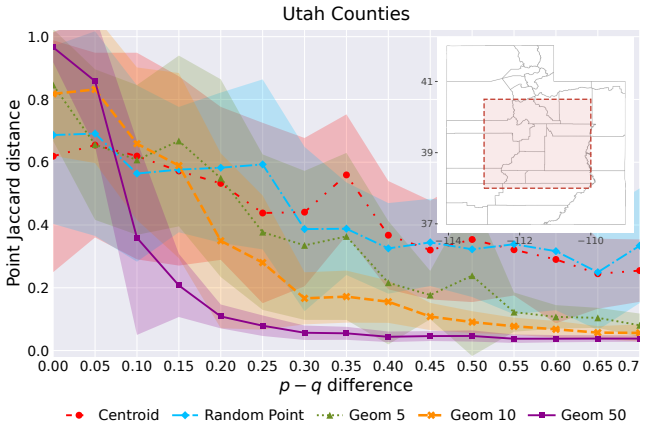

Figure 4: Utah with 29 counties. The inset (upper right) shows the state and the planted target rectangle (red dashed), spanning longitudes −113 to −110 and latitudes 38 to 40.5. The main plot shows Point Jaccard distance versus pq difference for each method at grid resolution 100, averaged over 20 trials with ±1 standard deviation bands.

their performance stabilizes as the pq difference increases. Specifically, Geom 10 begins to consistently recover the planted region at a pq difference of around 0.25, whereas Geom 50 achieves reliable detection at a pq difference as low as 0.15.

California. The counties in the state of California present a distinct challenge for spatial analysis. Although there are 69 counties, the state's shape is oblong and the counties are of vastly different sizes, introducing spatial heterogeneity and complicating detection. The scenario we consider, shown in Figure 5, has the target rectangle that separates several smaller counties in the Bay Area and even includes areas outside the state boundaries.

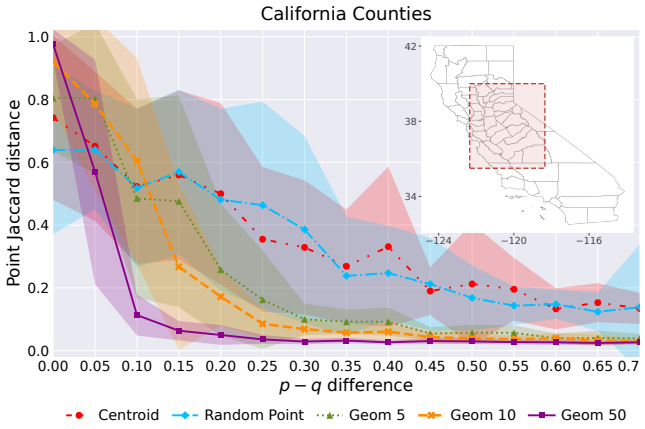

Figure 5: California with 69 counties of varying sizes. The inset (upper right) shows the state and the planted target rectangle (red dashed). The main plot shows Point Jaccard distance versus pq difference for each method at grid resolution 100.

The results, shown in Figure 5, highlight the effectiveness of our proposed method. The Geom 50 approach consistently achieves lower Point Jaccard distances across a range of pq differences, indicating greater statistical power. While the other methods, including the centroid approach, can achieve a small Point Jaccard distance, they have much more variation in what they find, demonstrated by much larger standard deviation bars.

All of USA. We evaluate our methods on a national scale, using all 3,711 counties in the continental United States; inset in Figure 6. The larger number of regions and the increased spatial coverage create a more stable statistical environment, allowing all methods to perform more reliably.

As shown in Figure 6, even the Centroid method shows improved performance on this scale, achieving a Point Jaccard distance of 0.2 when the pq difference is just 0.15, and a distance of 0.1 when the pq difference reaches 0.3. However, the proposed methods exhibit even more statistical power. For example, Geom 10 reaches a Point Jaccard distance of 0.2 with a pq difference as low as 0.05, and a distance of 0.1 with a pq difference of only 0.1.

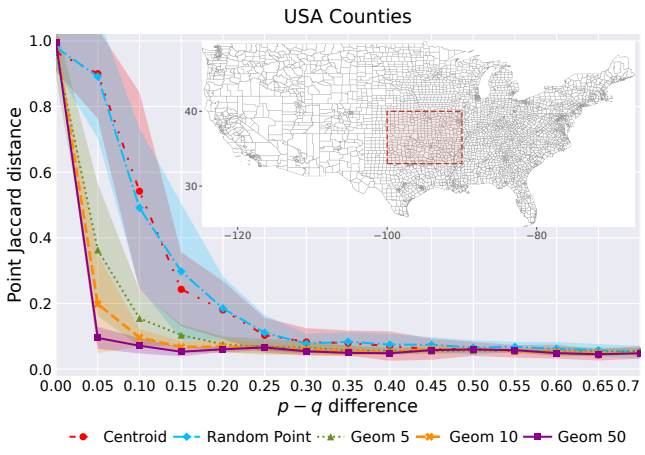

Figure 6: All 3,711 counties in the continental United States. The inset (upper right) shows the country and the planted target rectangle (red dashed), spanning longitudes −100 to −90 and latitudes 33 to 40. The main plot shows Point Jaccard distance versus pq difference for each method at grid resolution 100.

4.3 Effect of Target Rectangle Size

To further evaluate robustness, we conducted a different experiment to show how the methods are affected by the size of the planted region. We started by selecting a small rectangle and progressively increasing its size from 2% to 61% of the state of Georgia; see Figure 7. In this experiment, we fixed the pq difference at 0.4 to isolate the effect of the region size on the statistical power.

As the target rectangle becomes smaller, it contains fewer anomalous signals, making it more difficult to distinguish it from natural random variation elsewhere. The results in Figure 7 show that the Centroid and Random Point methods struggle across all region sizes, with performance degrading substantially for regions smaller than 12% of the state.

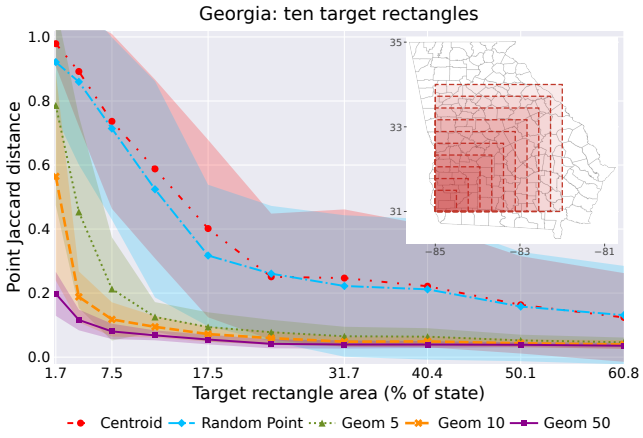

Figure 7: Point Jaccard distance as a function of the relative size of the planted target rectangle in Georgia, with $p-q$ fixed at 0.4 and averaged over 80 trials. The inset shows the ten nested target rectangles overlaid on Georgia counties; their areas range from about 2% to 61% of the state.

In contrast, our proposed methods (Geom 5 and Geom 10) demonstrate significantly higher robustness. With just 5 random points per region, regions as small as 8% of the state are still reliably detected (Point Jaccard distance below 0.2). With 10 random points, detection remains accurate even for regions as small as 4% of the state. This result highlights that adding even a few random points (instead of 1) in each region can greatly enhance the statistical power of the methods to detect significantly small spatial anomalies.

4.4 Comparison with FlexScan and Buchin etal.

We also evaluate our proposed method in counties in the state of Arkansas and compare its performance with the FlexScan[1] technique (a brown curve), and the area-based method of Buchin et al. [5] (a dark blue curve). FlexScan identifies a connected region of counties with elevated rates. The Arkansas dataset includes a moderate number of counties (75), each with fairly uniform shapes, making it a favorable setting for FlexScan.

As in previous experiments, we plant a rectangular anomaly, this time covering approximately 30% of the state; see Figure 8.

[1]FleXScan v3.1.2 software manual instructions [22]

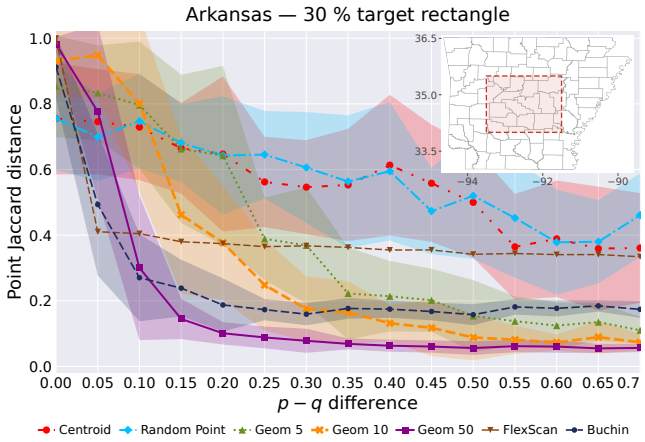

Figure 8: Point Jaccard distance between the planted target and the discovered region for Arkansas, with the target rectangle covering about 30% of the state (longitudes −93.5 to −91.5, latitudes 34 to 35.5).

The plot in Figure 8 shows how the methods perform as the pq difference increases. We observe that the performance of the Centroid, Random Point, and Geom 5, 10, 50 methods follows similar trends as in previous experiments. The centroid and random point methods are very noisy and inaccurate, converging to about a Point Jaccard distance 0.2 at a pq difference of 0.07, and performing worse on average at smaller pq values. In contrast, the Geom 5, 10, and 50 methods robustly and accurately identify the target rectangle, even at smaller pq differences. For example, Geom 50 achieves a Point Jaccard distance of 0.2 with only $pq = 0.1$, and improves to 0.1 at $pq = 0.25$.

FlexScan. While FlexScan performs reasonably, its Point Jaccard distance plateaus around 0.35–0.4 (above the shape floor of about 0.2), even as the pq difference increases. This appears to be an artificial implementation limitation, as the code limits the search for the potentially anomalous cluster to at most 15 regions.

To ensure a fair comparison, we repeated the experiment on the Arkansas dataset using a smaller target rectangle covering only 10% of the area, results shown in Figure 9. This setup is more challenging for both the Centroid and our proposed methods. Nevertheless, Geom 50 remains the best performer, achieving a Point Jaccard distance of approximately 0.2 (or less) for $pq \geq 0.25$. FlexScan, on the other hand, fails to achieve a Point Jaccard distance below 0.35, even as pq increases. Notably, for very small pq differences (≤ 0.1), FlexScan outperforms our methods. Still, this advantage is limited, as the Point Jaccard distance is still quite high (Jaccard distance ≈ 0.6), indicating that the overlap between the target and the discovered regions has some overlap on average, but not much.

Buchin. On the larger target (Figure 8) the area-based method of Buchin et al. [5] tracks Geom 50 closely, selecting nearly the correct scale (on average 1.07× the planted area; offset ∼6.0 km) even though it was given the planted size, and reaching a plateau within ∼0.1 of Geom 50. On the smaller target (Figure 9) the two methods separate: Buchin systematically oversizes to on average 1.61× planted (offset ∼11.4 km) and plateaus near 0.45, while Geom 50, whose continuous search adapts the scale, stays at 0.95× planted (offset ∼3.1 km) and reaches 0.16. Figure 10 shows these discovered rectangles geographically. Unlike the area-based method, our approach requires no user-supplied window size and is roughly 90× faster on Arkansas (Section 4.6, Table 2). A complementary head-to-head on *disk-shaped* planted targets, where pyScan's disk scanner can match the target exactly, is reported in Appendix B.

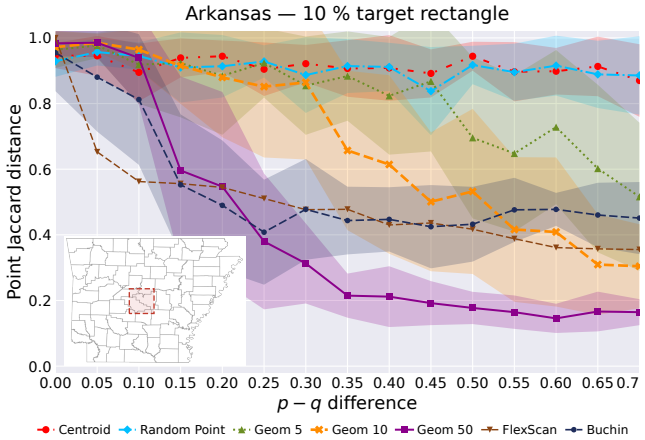

Figure 9: Point Jaccard distance between the planted and discovered regions for Arkansas, with the target rectangle covering about 10% of the state (longitudes −92.85 to −92.15, latitudes 34.4 to 35.1).

4.5 Ablation Study on Sampling and Similarity Measures

Finally, we consider two design choices: (1) the similarity metric used to compare the target and discovered regions, and (2) the sampling points strategy within each region. Our findings indicate that, while these choices introduce slight differences in performance, the selected default settings yield consistently strong results.

Our main evaluation metric, the *Point Jaccard distance*, is based on comparing point sets inside each region. Specifically, we sample $500n$ points from the n regions and compute the Jaccard distance between the sets associated with the target and discovered regions.

As an alternative, we could consider the *Area Jaccard distance*, which compares regions directly via geometric overlap. This metric is defined as one minus the ratio of the intersection area to the union area between the target and discovered regions. It provides a shape-based similarity score.

We also compare two strategies for sampling points within each region: *Uniform Sampling:* Each region receives an equal number of sampled points (e.g., 1, 5, 10, or 50). *Weighted Sampling:* Each region is guaranteed at least one point, and the remaining points are distributed proportionally to the region's baseline population. This aims to reflect a higher signal strength in denser regions.

While weighted sampling might seem more natural, it can lead to poor performance in sparse regions. For example, in the Georgia dataset, densely populated areas such as Atlanta dominate the sample, causing smaller or rural counties to receive few or no

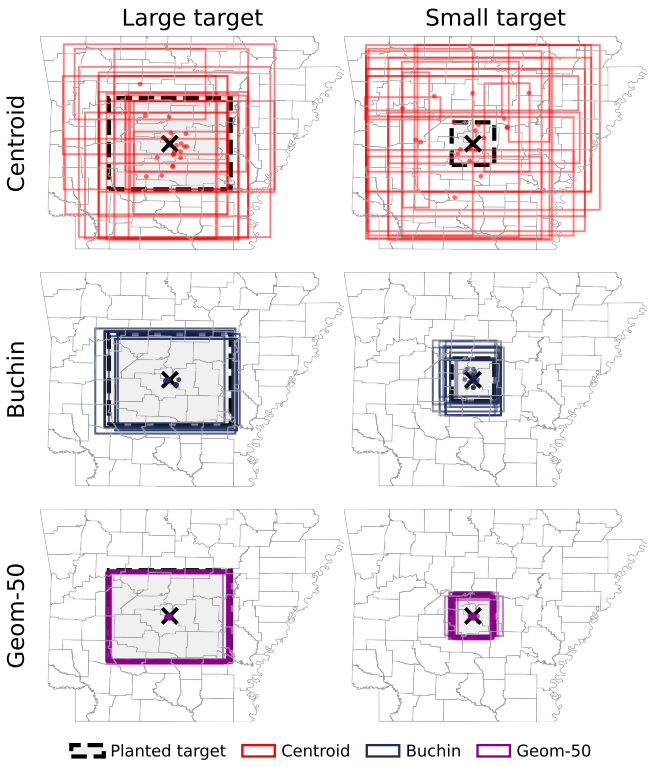

Figure 10: Discovered rectangular windows on Arkansas for both planted targets (black dashed) at $p - q = 0.5$, 20 trials each. Columns: larger target (30% of state) and smaller target (10% of state). Rows: Centroid (red), Buchin (navy), and Geom 50 (magenta). On the large target, Centroid oversizes to 1.41× planted (offset ∼36.1 km), Buchin selects nearly the correct scale at 1.07× (offset ∼6.0 km), and Geom 50 recovers the rectangle at 0.97× (offset ∼2.9 km). On the small target, Centroid degenerates to 9.93× planted (offset ∼59.4 km), Buchin overshoots to 1.61× (offset ∼11.4 km), and Geom 50 stays at 0.95× (offset ∼3.1 km). All pyScan-side methods use grid resolution 100.

points—diminishing the ability to detect target rectangles that span low-density areas.

Figure 11 shows all four combinations in Georgia counties, using Geom 50 on a single plot using a target rectangle covering about 30% of the area. It again shows the two uniform sampling methods performing slightly better for small values of the pq difference.

Among the top 4 plots, the first row uses the Point Jaccard distance and the second row uses the Area Jaccard distance. The first column uses uniform point sampling, and the second uses weighted point sampling. While there are some differences among these four plots, all give the same general effect. Perhaps the only significant signal is that under uniform point sampling, Geom 5 and Geom 10 more consistently recover the target region for pq difference in the range of 0.15 to 0.25, when compared to using weighted sampling.

4.6 Runtime

Converting a problem with n input points to $50n$ (as we did in Geom 50) points could lead to a significant computational slowdown, changing problems from tractable to intractable. Although naive algorithms for computing spatial scan statistics over rectangles on n points scale with $O(n^4)$ or $O(n^5)$, there are improvements that reduce this to near quadratic in n; Agarwal *et.al.* [3] show how to compute this in $O(n^2 \log n)$ time. An efficient version of this algorithm is implemented in pyScan, which we employ in this paper. Even with this optimized implementation, a 50× increase in data could imply a theoretical 2,500× slowdown. In practice, this worst case does not occur. pyScan's `max_subgrid` scans a fixed-resolution grid (here 100×100), so its running time depends on the grid size, not on how many input points there are. Increasing the points per region therefore has little effect on runtime. Concretely, going from Arkansas ($n = 75$) to the USA ($n = 3{,}711$) is a 50× increase in regions, yet the scan time grows only about 1.2× (Tables 1 and 2).

All runtimes below report wall-clock for a single cluster discovery at $p = 0.6$, $q = 0.2$, measuring only the scan step (case generation and data loading excluded). pyScan uses grid resolution 100; Buchin sweeps its size grid around the planted size. FlexScan's number additionally includes a Monte-Carlo significance test that the other methods do not perform.

	Geom 1	Geom 5	Geom 10	Geom 50	Buchin	FlexScan
Runtime (s)	0.29	0.29	0.29	0.33	114.0	1109
Std	0.001	0.001	0.002	0.002	4.4	–

Table 1: Runtime (second) on USA counties ($n = 3{,}711$), one cluster discovery, scan step only, grid resolution 100, $p = 0.6$, $q = 0.2$. Geom k and Buchin: mean of 5 independent runs. Buchin sweeps its nine-point size grid $\{0.5, 0.7, 0.85, 0.95, 1.0, 1.05, 1.15, 1.3, 1.5\}\times$ the planted size.

Table 1 shows that pyScan's scan time is dominated by the fixed-resolution grid scan, so Geom k for $k \in \{1, 5, 10, 50\}$ all complete one discovery in roughly 0.3 seconds on the USA dataset ($n = 3{,}711$ regions). While we demonstrate this with pyScan, other efficient scanning approaches, like those by Neill and Moore [16], would also keep this manageable. Buchin's rectangle scanner takes 114 seconds per discovery — about 350× slower than Geom 50 — because it sweeps a nine-point size grid and performs a translation search at each size. FlexScan takes 1,109 seconds (about 18 minutes), though that number includes a Monte-Carlo significance test the other methods do not perform. As discussed in Section 2.1, FlexScan also presents interpretability challenges on large datasets, returning multiple clusters that are not straightforward to reconcile.

To complement our large-scale analysis, we also evaluated runtimes on the smaller Arkansas dataset, using a planted region that covers approximately 30% of the state. This setup aligns with prior evaluation studies [23, 24] using FlexScan. Geom 50 completes a single discovery in approximately 0.28 seconds; Buchin's rectangle scanner takes 25 seconds (about 90× slower), and FlexScan takes 8 seconds, including its Monte-Carlo significance test. These results, shown in Table 2, demonstrate that our method's runtime advantage holds across both small and large region counts.

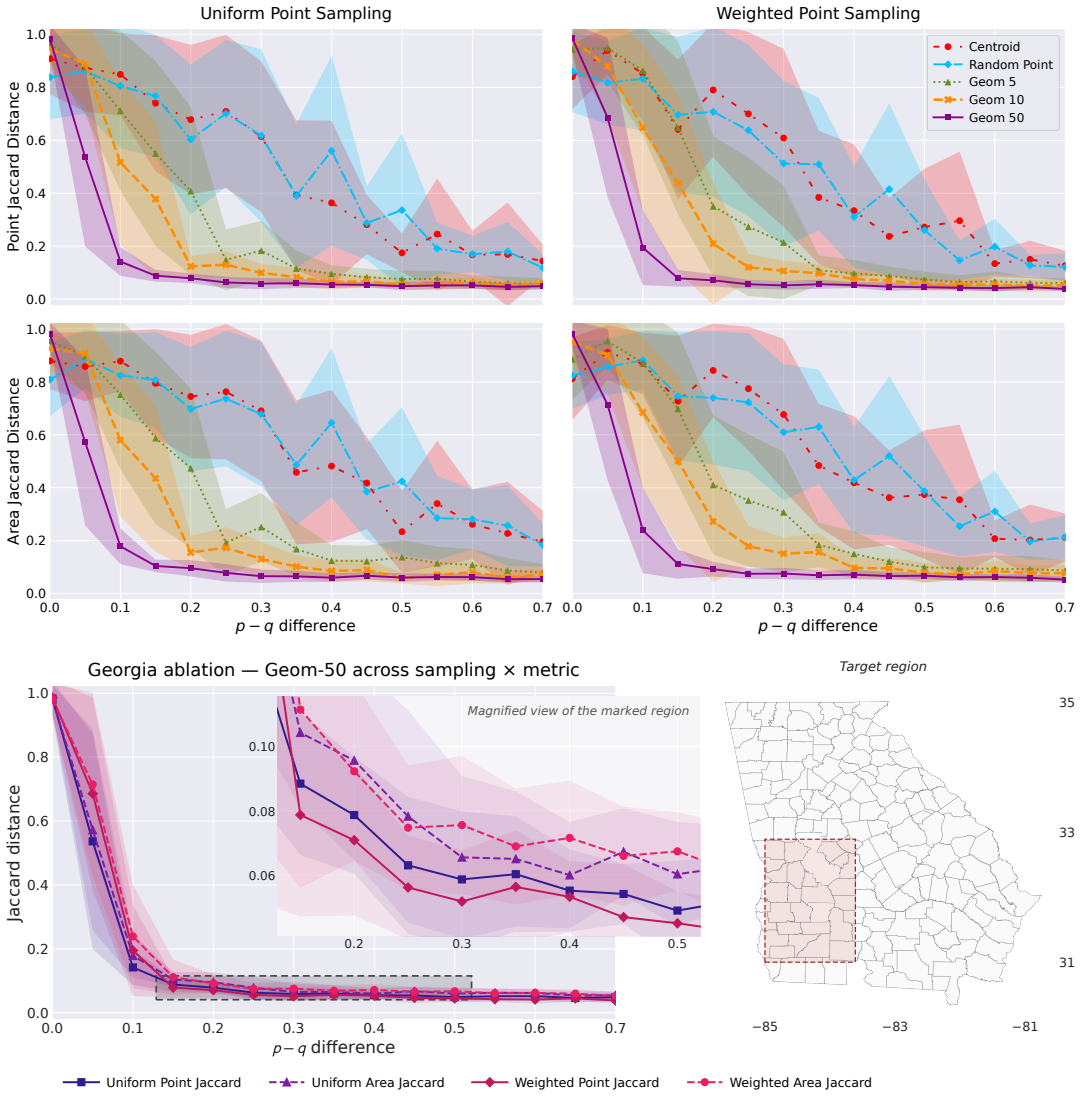

Figure 11: Georgia ablation at grid resolution 100, on Georgia counties with a planted region covering approximately 30% of the state. Top: the 2×2 grid shows Point Jaccard distance (top row) and Area Jaccard distance (bottom row) under Uniform Point Sampling (left column) and Weighted Point Sampling (right column) across the five methods. Bottom: focused comparison of Geom 50 across the four (sampling, metric) settings, with the magnified panel zooming into $p - q \in [0.15, 0.50]$ where the four settings separate; the planted target region is shown on the Georgia map on the right.

	Geom 1	Geom 5	Geom 10	Geom 50	Buchin	FlexScan
Runtime (s)	0.21	0.26	0.27	0.28	25.3	8.00
Std	0.015	0.005	0.002	0.002	0.12	–

Table 2: Runtime (second) on Arkansas counties ($n = 75$), one cluster discovery, scan step only, grid resolution 100, $p = 0.6$, $q = 0.2$. Geom k and Buchin: mean of 5 independent runs; Buchin uses the same nine-point size grid as in Table 1.

Finally, while not directly used in this study, pyScan also offers methods that compute the approximately optimal spatial scan statistic. These employ algorithms that involve subsampling data and other methods to scan them efficiently. In particular, these methods use coresets [20], which first reduce the dataset problem to a size that depends only on the desired accuracy guarantee. As a result, the initial size of the data set no longer becomes a runtime constraint (other than the time to store and sample from it), only the desired degree of accuracy. In the context of these methods, the proposed approach will not meaningfully impact the runtime.

5 Convergence

Our experiments demonstrate that sampling multiple points per region improves the statistical power, enabling more accurate recovery of true anomalous regions. However, we have not yet provided a theoretical explanation of why these methods work. In this section, we offer a stylized analysis to provide such insight. As with any stylized analysis, it will be an imperfect model of real-world settings, but we hope it provides some insight nonetheless.

There are two key components to understanding how accuracy improves with the number k of sample points per region in Geom k. First, how accurately we can approximate the function ϕ, which the scan statistics algorithm seeks to optimize. Second, how this approximation translates to the quality of the recovered region, measured using Jaccard distance d_{JAC}. We address the second issue first. The scan statistic problem is a non-convex optimization problem with the potential for many local minimum (e.g., see conditional hardness results [4, 14]), so unless we make some structural assumptions about the problem, even small approximation errors in ϕ could lead to a significant different optimal shape, resulting in a large change in Jaccard distance. To address this, we propose to consider *(ε, γ)-stable shapes*. An optimal shape $S^* \in \mathcal{S}$ is (ε, γ)-stable if for any $S' \in \mathcal{S}$ such that $d_{\text{JAC}}(S^*, S') > \gamma$ it holds that $\phi(S^*) - \phi(S') > \varepsilon$. This stability assumption implies that any shape with a score within ε of the optimum must be within γ in Jaccard distance.

While this assumption cannot be guaranteed for all real-world scenarios, in our simulated setting – with its controlled random generation process on a known and fixed S^* – we can satisfy that the optimal region S^* is (ε, γ)-stable for some choice of ε and γ; at least with a large enough sample size and pq-distance. Thus, for our theoretical analysis, we assume to be seeking an (ε, γ)-stable shape; we will show that we can achieve ε-approximation in the cost ϕ, and this will imply we also obtain γ-approximation in the Jaccard distance.

To ensure that the recovered region $(\hat{S})$ is close to the true optimal region (S^*), we require that the difference in their scan statistic score is at most ε, that is

$$|\phi(\hat{S}) - \phi(S^*)| \leq \varepsilon.$$

In other words, we want the discovered region $\hat{S}$ to approximate the optimal region S^*, within a small margin of error in term of the score function ϕ. To apply this in our setting, we need one additional concept, since the regions are imbalanced in size. Hence, we may need more points from the larger regions (e.g., more samples from Texas than Delaware in a US state dataset). To allow for analysis of our proposed algorithm, we consider an input of a set of regions $\mathcal{R}$ to be *κ-balanced* if $\max_{R \in \mathcal{R}} m(R) \leq \kappa \min_{R \in \mathcal{R}} m(R)$ and $\max_{R \in \mathcal{R}} b(R) \leq \kappa \min_{R \in \mathcal{R}} b(R)$. We can now state the following theorem.

Theorem 5.1. *Consider an (ε, γ)-stable anomalous shape S^* in a κ-balanced set of n regions. If we use Geom k to sample $k = O(\kappa/n\varepsilon^2)$ uniformly distributed points from each region and then run a scan statistics algorithm to obtain a shape $\hat{S}$, then with a constant probability we can guarantee $d_{\text{JAC}}(S^*, \hat{S}) \leq \gamma$.*

Proof. We rely on a probabilistic guarantee established in prior work [14, 15] that we can ensure an accurate estimate of ϕ under sampling. In particular, if we consider a random sample $X' \subset X$ of size $|X'| = O(1/\varepsilon^2)$, then with constant probability, the optimal scan statistic computed on X' will have a solution $\hat{S} = S'^*$ so that $|\phi(S'^*) - \phi(S^*)| \leq \varepsilon$, where S^* is the optimal solution of the full data set X.

To apply these results in our settings, we treat the input X as a continuous distribution – in particular, a uniform measure over each input region. In our application, X represents either the measured value m or the baseline value b; where the density assigned to each region z is given by $m(z)$ or $b(z)$, respectively. There is nothing in the above results [14, 15] that prevents X from being a continuous distribution, as long as we can draw random samples from it. Since we can draw uniformly from each shape file region, this modeling assumption is justified. The remaining challenge is that the required $O(1/\varepsilon^2)$ samples must be drawn from the full dataset, proportionally to the baseline b (and likewise m). This means that if there are n regions, we require $kn = O(1/\varepsilon^2)$ total sample, $k = O(1/n\varepsilon^2)$ points – indicating as we subdivide into a larger number of regions n, the total number of samples stays fixed, and thus the number of samples needed per regions decreases.

However, this $k = O(1/n\varepsilon^2)$ points per region property may not always be true, since if the regions are imbalanced in size, then we may need more points from the larger regions (e.g., more samples from Texas than Delaware in a US state dataset). Then, if we sample $k = O(\kappa/n\varepsilon^2)$ points per region, it will over-sample from some smaller regions, but guarantee to obtain the equivalent of the requisite (at most κ/ε^2) number of samples even in the largest regions.

Piecing together these components and transferring ε-error on ϕ to γ-Jaccard error via the (ε, γ)-stable assumption completes the proof. □

6 Discussion

Our method achieves fast runtimes and accuracy across datasets, including large-scale experiments. As shown in Table 2, even on smaller datasets like Arkansas, where FlexScan performs relatively well (8 seconds), our approach completes in under 1 second. This efficiency and consistency shows practicality of our approach for both small scale and large scale spatial analysis.

This paper uses a model where the data in each region z is uniformly mapped to the area in that region through a sample. However, the human population is certainly not distributed uniformly in spatial regions, and one could think of other ways to distribute the measured and baseline values. This could make models more realistic in baseline values, but perhaps less realistic in measured values. Our view in this paper is that when data are aggregated to regions, this fine-grained spatial information is lost, and uniform is a reasonable approach, but future work may address this more carefully. We note that the Section 4.5 ablation tests a population-weighted alternative ("Weighted Sampling"); it does not outperform uniform sampling, supporting that the uniform-within-region assumption is conservative rather than fragile. [Foad: Added per GeoInf-R2: reviewer questioned the uniform-within-region assumption and asked for either expanded Section 6 discussion or a population-weighted real-data experiment. The Section 4.5 Weighted Sampling

ablation already somehow in some datasets empirically refutes the concern.]

There are also other ways we could assess statistical power. This could involve how we evaluate similar rectangles, how we distribute our $50n$ samples, or experiments other than changing the pq difference or the target region size. We informally explored other ideas beyond the ones reported in this paper, and they either gave results similar to the ones we showed or were far less reliable in measurement. The paper presents the ones we feel most clearly demonstrate the merits of the main conceptual frameworks. Nevertheless, other ways one could use these ideas to improve spatial scan statistics may arise from the principles explained in this paper.

7 Conclusion

We introduce a simple yet effective method to converting region-based inputs to point-based representation, enabling efficient algorithms for spatial scan statistics. Our approach—sampling multiple points uniformly within each region—significantly improves statistical power compared to standard techniques that rely solely on region centroids. Through controlled planted-region experiments, we demonstrated that our method can recover anomalous regions under more subtle effect sizes while remaining computationally tractable.

Leveraging efficient implementation such as pyScan, we showed that even with increased point counts (e.g., Geom 50), runtime remains practical across both small and large datasets. This supports the scalability of our method without sacrificing precision or speed.

Given its ease of implementation, superior performance, and compatibility with existing algorithms, we recommend this sampling based approach as the default way to convert region-based input to point-based representations for spatial scan statistics.

In contrast, the standard method in SatScan (represented by the Centroid method) has less power, and connected-component-based methods such as FlexScan are slower and do not show as much power in our experiments.

References

[1] Ali Abolhassani and Marcos O Prates. 2021. An up-to-date review of scan statistics. *Statistic Surveys* 15 (2021), 111–153.

[2] Deepak Agarwal, Andrew McGregor, Jeff M Phillips, Suresh Venkatasubramanian, and Zhengyuan Zhu. 2006. Spatial scan statistics: approximations and performance study. In *Proceedings of the 12th ACM SIGKDD international conference on Knowledge discovery and data mining*. 24–33.

[3] Deepak Agarwal, Jeff M. Phillips, and Suresh Venkatasubramanian. 2006. The Hunting of the Bump: On Maximizing Statistical Discrepancy. *SODA* (2006), 1137–1146.

[4] Arturs Backurs, Nishanth Dikkala, and Christos Tzamos. 2016. Tight Hardness Results for Maximum Weight Rectangles. In *43rd International Colloquium on Automata, Languages, and Programming (ICALP 2016)*, Vol. 55. Schloss Dagstuhl, Dagstuhl, Germany, 81.

[5] Kevin Buchin, Maike Buchin, Marc van Kreveld, Maarten Löffler, Jun Luo, and Rodrigo I. Silveira. 2012. Processing aggregated data: the location of clusters in health data. *GeoInformatica* 16, 3 (2012), 497–521. doi:10.1007/s10707-011-0143-6

[6] Michael R Desjardins, Alexander Hohl, and Eric M Delmelle. 2020. Rapid surveillance of COVID-19 in the United States using a prospective space-time scan statistic: Detecting and evaluating emerging clusters. *Applied geography* 118 (2020), 102202.

[7] Joseph Glaz and Markos V Koutras. 2024. *Handbook of scan statistics*. Springer.

[8] Tony H Grubesic, Ran Wei, and Alan T Murray. 2014. Spatial clustering overview and comparison: Accuracy, sensitivity, and computational expense. *Annals of the Association of American Geographers* 104, 6 (2014), 1134–1156.

[9] Mingxuan Han, Michael Matheny, and Jeff M Phillips. 2019. The kernel spatial scan statistic. In *Proceedings of the 27th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems*. ACM, New York, NY, USA, 349–358.

[10] Martin Kulldorff. 1997. A spatial scan statistic. *Communications in Statistics-Theory and methods* 26, 6 (1997), 1481–1496.

[11] Martin Kulldorff. 2022. *SatScan User Guide* (10.1 ed.). http://www.satscan.org/.

[12] Martin Kulldorff, Lan Huang, Linda Pickle, and Luiz Duczmal. 2006. An elliptic spatial scan statistic. *Statistics in medicine* 25, 22 (2006), 3929–43.

[13] Michael Matheny. 2024. *pyScan*. https://github.com/michaelmathen/pyscan.

[14] Michael Matheny and Jeff M. Phillips. 2018. Computing Approximate Statistical Discrepancy. *ISAAC* 123 (2018), 7:1–7:14.

[15] Michael Matheny, Raghvendra Singh, Liang Zhang, Kaiqiang Wang, and Jeff M. Phillips. 2016. Scalable Spatial Scan Statistics Through Sampling. In *SIGSPATIAL*. ACM, New York, NY, USA, 1–10.

[16] Daniel B. Neill and Andrew W. Moore. 2004. Rapid Detection of Significant Spatial Clusters. In *KDD*. ACM, New York, NY, USA, 256–265.

[17] Daniel B. Neill, Andrew W. Moore, and Gregory F. Cooper. 2006. A Bayesian Spatial Scan Statistic. In *NIPS*. MIT Press, Cambridge, MA, USA, 1003–1010.

[18] Mallory Nobles, Ramona Lall, Robert W Mathes, and Daniel B Neill. 2022. Presyndromic surveillance for improved detection of emerging public health threats. *Science Advances* 8, 44 (2022), eabm4920.

[19] Ganapati P Patil and Charles Taillie. 2004. Upper level set scan statistic for detecting arbitrarily shaped hotspots. *Environmental and Ecological statistics* 11 (2004), 183–197.

[20] Jeff M Phillips. 2017. Coresets and sketches. In *Handbook of discrete and computational geometry*. Chapman and Hall/CRC, 1269–1288.

[21] Shino Shiode. 2011. Street-level spatial scan statistic and STAC for analysing street crime concentrations. *Transactions in GIS* 15, 3 (2011), 365–383.

[22] Tetsuji Yokoyama Takahashi and Toshiro Tango. 2023. *FleXScan: Software for the Flexible Scan Statistics*. https://sites.google.com/site/flexscansoftware/.

[23] Toshiro Tango and Kunihiko Takahashi. 2005. A flexibly shaped spatial scan statistic for detecting clusters. *International journal of health geographics* 4 (2005), 1–15.

[24] Toshiro Tango and Kunihiko Takahashi. 2012. A Flexible Spatial Scan Statistic with a Restricted Likelihood Ratio for Detecting Clusters. *Statistics in Medicine* 31, 30 (2012), 4207–4218. doi:10.1002/sim.5478

[25] Yiqun Xie, Shashi Shekhar, and Yan Li. 2022. Statistically-robust clustering techniques for mapping spatial hotspots: A survey. *ACM Computing Surveys (CSUR)* 55, 2 (2022), 1–38.

Acknowledgments

DM and MM work done while at the University of Utah. Part of the work by JP was completed while visiting ScaDS.AI, University of Leipzig, and MPI for Math in the Sciences.

This research was supported in part by NSF grants CCF-2115677 and IIS-2311954.

Appendix A Implementation and pyScan

We include these listings as a reproducibility aid: together they reproduce, in under thirty lines of Python, the full Geom 50 pipeline used throughout Section 4 on any input shapefile, so a reader can replicate our results without reconstructing the pyScan calls from the body of the paper.

We illustrate how to run pyScan on U.S. counties using the Geom 50 setting. First, we load a shapefile of U.S. counties.

```
import pyscan
import geopandas as gpd
import numpy as np
import random
from shapely.geometry import Point, Polygon
import matplotlib.pyplot as plt

# Load shapefile of US counties
us_df = gpd.read_file("us_county.shp").to_crs("EPSG:4326")

# Check the first few entries
print(us_df.head())
```

Listing 1: Python Code to Load Shapefile

Next, we define a target rectangle over the state of interest:

```
# Define target rectangle
target_rectangle = Polygon([(-100, 33), (-100, 40), (-90, 40),
      (-90, 33)])
```

Listing 2: Defining a Target Rectangle Using Polygon

We generate 50 random points inside each region and assign all of them to the baseline set. Then, we construct the measured set probabilistically: points inside the target rectangle are included with probability 0.4, and those outside with probability 0.2.

```
# Sample 50 points from each region
n_geom = 50
sampled_points = []

for i, row in us_df.iterrows():
    polygon = row['geometry']
    min_x, min_y, max_x, max_y = polygon.bounds
    region_points = []

    while len(region_points) < n_geom:
        # Generate a random point within the bounding box
        random_point = Point([random.uniform(min_x, max_x),
      random.uniform(min_y, max_y)])
        if random_point.within(polygon):
            region_points.append([random_point.x,
      random_point.y, i])

    sampled_points.append(region_points)

sampled_points = np.vstack(sampled_points)
```

Listing 3: Python Code to Generate 50 Random Points Inside Each Region

```
# Create baseline and measured sets to test the algorithm using
      p=0.4 (inside target), q=0.2 (outside)
baseline = []
measured = []

for point in sampled_points:
    prob = random.random()
    # WPoint(weight, x, y, value): defines a weighted point
      with attribute for pyScan
    baseline.append(pyscan.WPoint(1.0, point[0], point[1], 1.0))
    pt = Point(point[0], point[1])

    if target_rectangle.contains(pt):
        if prob <= 0.4:
            measured.append(pyscan.WPoint(1.0, point[0],
      point[1], 1.0))
    else:
        if prob <= 0.2:
            measured.append(pyscan.WPoint(1.0, point[0],
      point[1], 1.0))
```

Listing 4: Assigning Points to Baseline and Measured Sets Using Probabilistic Inclusion

We use the Kulldorff scan statistic (Poisson model) to detect anomalous regions. Then we pass the measure and baseline lists to pyscan.Grid(). We then apply pyscan.max_subgrid() to find the most anomalous rectangular subregion, stored in the variable subgrid. This rectangle represents the discovered rectangle.

```
# Run pyScan to find most anomalous rectangle
rect_f = pyscan.KULLDORF
grid = pyscan.Grid(100, measured, baseline)
subgrid = pyscan.max_subgrid(grid, rect_f)
rect = grid.toRectangle(subgrid)
```

Listing 5: Python Code to Run pyScan with Geom 50 Sampling

To visualize the result, we plot the detected rectangle over the sampled points:

```
# Plot discovered rectangle
plt.scatter(sampled_points[:, 0], sampled_points[:, 1], s=2)
plt.gca().add_patch(
    plt.Rectangle((rect.lowX(), rect.lowY()),
                  rect.upX() - rect.lowX(),
                  rect.upY() - rect.lowY(),
                  edgecolor='red', facecolor='none')
)
plt.axis('off')
plt.show()
```

Listing 6: Python Code to Plot the Detected Rectangle Over Sampled Points

Full examples and further documentation for pyScan are available at:
https://mmath.dev/pyscan

A.1 Choice of k

[Foad: I added this section] [Jeff: I am not sure this adds anything, and it does not align with the theorem. The size k should depend on n – the number of regions.] [Foad: I added the $1/\sqrt{k}$ as dashed line to k-sweep plot,]

To empirically justify the abstract's recommendation of $k = 20$–50 samples per region, we sweep k from 2 to 100 on all six datasets. Here k is the number of uniformly random points drawn from

each region's polygon, so a state with n regions contributes kn total points to the scan input. We fix the rate contrast at the pq difference = 0.15 and report the Point Jaccard distance averaged over 20 trials per setting (seed = 7). We chose $pq = 0.15$ because it sits in the transition regime where small-k methods still fail and large-k methods clearly succeed: at much smaller pq every method is stuck near chance on all datasets, while at much larger pq even $k = 2$ already detects the planted region, so the k-dependent diminishing-returns trend is most visible at this value across all six datasets simultaneously.

Figure 12 shows the result. Point Jaccard distance drops sharply until $k \approx 20$, after which returns diminish on every dataset—from the 75-county Arkansas case up to the 3,108-county continental United States. This motivates our $k = 20$–50 recommendation: large enough to capture the within-region geometry, small enough to keep the input to pyScan tractable.

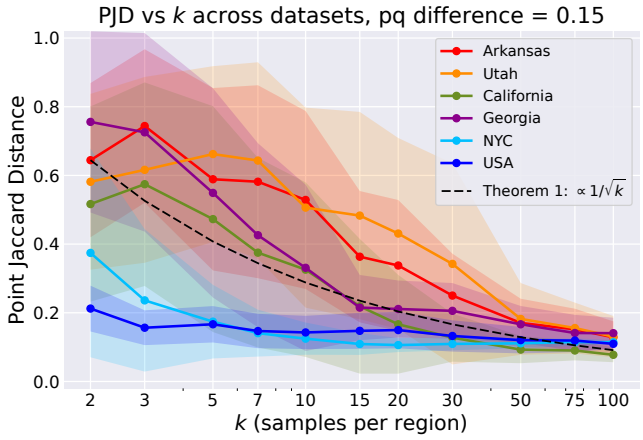

Figure 12: Point Jaccard distance as a function of k (uniformly random samples per region), at a fixed pq difference of 0.15, across all six datasets. $pq = 0.15$ is the transition regime where small k still fails and large k succeeds, so the diminishing-returns shape is visible on every dataset. Each curve is the mean over 20 trials with random seed 7; shaded bands show ± 1 standard deviation. Returns diminish past $k \approx 20$, motivating the abstract's $k = 20$–50 recommendation. The black dashed line overlays Theorem 5.1's predicted asymptote $d_{\text{Jac}} \propto 1/\sqrt{k}$, calibrated to Arkansas at the smallest sampled k; the empirical curves track the predicted decay shape past $k \approx 10$. [Foad: Per GeoInf-R1 (C2): the fine sweep over k is here, which now plots $k \in \{2, 3, 5, 7, 10, 15, 20, 30, 50, 75, 100\}$ on all six datasets ($k = 1$ is the Centroid baseline already in every main figure). The dashed black line overlays Theorem 5.1's predicted $1/\sqrt{k}$ asymptote, calibrated to Arkansas at the smallest sampled k. The empirical curves track the predicted decay shape past $k \approx 10$, where the (ε, γ)-stable regime kicks in.]

Appendix B Disk Cluster Recovery vs. Buchin

We repeat the head-to-head comparison with the area-based method of Buchin et al. [5] using *circular* planted targets. Because pyScan's disk scanner can match a disk target exactly, this setting isolates scale and position recovery with no shape-family floor. We plant disks of radius approximately 67 km and 44 km, centered in the state, and follow the same protocol as the rectangular comparison in Section 4.4: $q = 0.2$, Point Jaccard distance averaged over 20 trials with ± 1 standard deviation bands, Buchin given its standard nine-point size grid, and Centroid included as a lower reference.

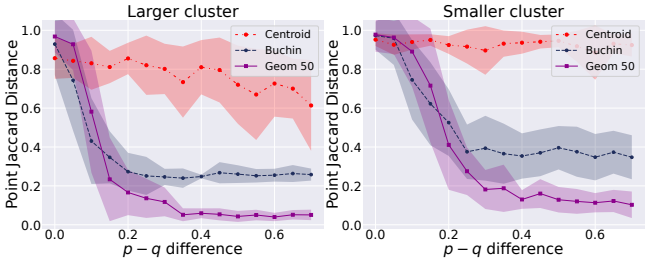

Figure 13: Circular cluster recovery on Arkansas counties: Point Jaccard distance vs. pq difference for planted disks of radius ~ 67 km (left) and ~ 44 km (right). Geom 50 (blue) recovers the correct radius at both sizes; the Buchin method (green) oversizes by ~ 20–30% regardless of scale, giving a near-constant accuracy gap. Centroid (gray) degenerates under an unbounded disk search. Bands show ± 1 standard deviation over 20 trials.

Figure 13 shows the recovery curves; the pattern mirrors the rectangular case. On the larger disk the Buchin method is close to the correct radius (1.17× planted); on the smaller disk it oversizes more (1.25×), where its discrete grid adapts less well. Geom 50 recovers the correct radius at both sizes (1.02×), so the separation concentrates on the smaller, harder target. Buchin's residual oversizing occurs under its own nine-point size-selection protocol with the correct scale among the candidates, so it stems from the area-weighted score itself rather than any imposed restriction on the size search. Centroid degenerates under an unbounded disk search on one point per county, confirming its unsuitability as anything but a lower reference.

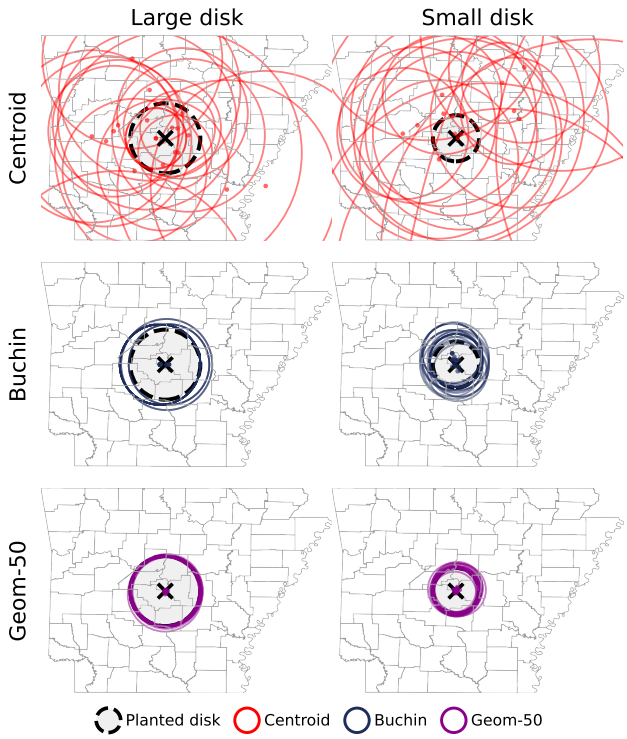

Figure 14: Discovered disk windows on Arkansas for both planted disk targets (black dashed) at $p-q=0.5$, 20 trials each, planted center $(-92.5,\ 34.75)$. Columns: large disk (radius ~ 67 km) and small disk (radius ~ 44 km). Rows: Centroid (red), Buchin (navy), and Geom 50 (magenta). On the large disk, Centroid degenerates to $2.17\times$ the planted radius (offset ~ 83.8 km), Buchin selects nearly the correct radius at $1.16\times$ (offset ~ 7.2 km), and Geom 50 recovers the disk at $1.01\times$ (offset ~ 1.8 km). On the small disk, Centroid degenerates to $6.57\times$ planted (offset ~ 205.2 km), Buchin oversizes to $1.25\times$ (offset ~ 10.8 km), and Geom 50 stays at $1.03\times$ (offset ~ 3.6 km). Geom 50 recovers the correct window at the right scale automatically at both sizes; the fixed-scale Buchin method is pulled toward an oversized window even when handed a grid containing the exact answer. All pyScan-side methods use grid resolution 100.

Figure 14 shows the discovered windows geographically. Geom 50 matches the Buchin method's accuracy on easy (large) targets, exceeds it on hard (small) targets for both window shapes, and—unlike the area-based method—requires no user-supplied window size. We note that pyScan's exact disk scan enumerates candidate disks through triples of points and becomes more expensive at national scale, so we present the disk comparison as an accuracy result on Arkansas rather than a runtime claim.[2]

[2]All windows are defined in EPSG:4326 lon/lat degree-Cartesian coordinates; the resulting geographic ellipticity (1° lon ≈ 91 km vs. 1° lat ≈ 111 km at this latitude) applies uniformly to all methods. The Buchin method's circular window is rendered as an inscribed regular polygon; our overlap measure treats it as a true circle, a ∼2% area difference in its favor. Centroid runs on an independent random stream; comparisons rely on mean-over-trials equivalence, not paired trials.